AI OPERATIONS (AIOps) · MODULE 4

Serving with MLflow

Module 4 · Deck 2 of 4
pyfunc models · signatures · Model Registry aliases · `mlflow models serve`

Third-year Data Science · CPU-only · all code runs on a laptop

Learning outcomes for this deck

Package a model as an MLflow pyfunc with a signature and input example

Register versions in the Model Registry and manage them with aliases

Serve a registry model with one command and query /invocations

Promote & roll back by moving an alias — and explain why a restart is needed

Compare MLflow serving with hand-written FastAPI (what you gain, what you lose)

Gate promotions on fresh data and observe a challenger with shadow traffic

Recall from Module 1 → what is new today

Module 1 (tracking)

- mlflow.log_param / log_metric
- Compare runs in the UI
- Log a model artifact
- Question answered: 'which run was best?'

Today (serving)

- Wrap the model as pyfunc + signature
- Register versions, set aliases
- Serve by registry URI
- Question answered: 'which build is live and how do I change it safely?'

Why it matters

The Registry turns 'a file on someone's laptop' into a governed, versioned asset with an audit trail and a one-line rollback.

The MLflow serving pipeline



Everything the server needs is stored with the model: code wrapper, artifacts, dependency list and the input/output schema. That is why `mlflow models serve` needs no extra code.

1

Packaging: pyfunc + signature

A universal serving interface and an enforceable contract

What is a pyfunc model?

python_function flavor: MLflow's universal interface — `predict(DataFrame) -> DataFrame/array`

Any library (sklearn, XGBoost, PyTorch, custom code) can be exposed through it

Custom `PythonModel`: load artifacts in `load_context`, implement `predict` — lets you return label + score + version

Signature: typed input and output columns, inferred from example data, enforced at serving time

input_example: stored sample request; MLflow validates it against the signature at log time

A custom pyfunc that wraps our joblib bundle

```python
class ChurnPyfunc(mlflow.pyfunc.PythonModel):
    """Wraps the joblib bundle so the served model returns probability + label + model version."""
    def load_context(self, context):
        b = joblib.load(context.artifacts["bundle"])
        self.m, self.cols, self.ver = b["model"], b["feature_columns"], b["version"]
    def predict(self, context, model_input, params=None):
        X = pd.DataFrame(model_input)[self.cols]
        p = self.m.predict_proba(X)[:, 1]
        return pd.DataFrame({"churn_probability": p.round(4), "churn": (p >= .5).astype(int),
                             "model_version": self.ver})
```

Output is a DataFrame with probability, label and model_version, so every served answer is traceable (same idea as Deck 1).

Log, sign and register — one call

```python
out_example = pd.DataFrame({"churn_probability": [0.1], "churn": [0], "model_version": ["v1"]})
sig = infer_signature(Xs.head(5), out_example)     # explicit input/output schema

with mlflow.start_run(run_name=f"churn-{ver}"):
    mlflow.log_param("algorithm", type(b["model"]).__name__)
    mlflow.log_metric("train_accuracy", acc)       # NOTE: in-sample! gate on fresh data later
    info = mlflow.pyfunc.log_model(
        name="model", python_model=ChurnPyfunc(),
        artifacts={"bundle": bundle},              # the joblib file travels with the model
        signature=sig, input_example=Xs.head(2),
        pip_requirements=["scikit-learn", "pandas", "joblib", "numpy"],
        registered_model_name="churn-model")       # creates version N of churn-model
```

Features are cast to float64 so JSON ints/floats are both accepted. Pin pip_requirements: the serving environment is rebuilt from them.

2

The Model Registry

Versions, aliases and a one-line rollback

Versions and aliases

Versions

- Immutable, numbered: churn-model v1, v2, …
- Each points to a logged run + artifacts
- Never edited — you create a new version

Aliases

- Movable names: champion, challenger
- Clients/servers refer to models:/churn-model@champion
- Promote = move `champion` to a new version
- Rollback = move it back
- Replaces the older, deprecated 'stages' (Staging/Production)

Aliases from code

```python
from mlflow import MlflowClient
client = MlflowClient()                    # tracking URI: sqlite:///mlflow.db (Registry needs a DB)

versions = client.search_model_versions("name='churn-model'")
client.set_registered_model_alias("churn-model", "champion",   1)
client.set_registered_model_alias("churn-model", "challenger", 2)

client.get_registered_model("churn-model").aliases     # {'champion': 1, 'challenger': 2}
client.get_model_version_by_alias("churn-model", "champion").version

# load the same model URI anywhere
model = mlflow.pyfunc.load_model("models:/churn-model@champion")
model.predict(df_of_8_features)

# environment:  export MLFLOW_TRACKING_URI=sqlite:///mlflow.db
```

3

Serving

One command, a generated REST API

Serve the champion

```bash
export MLFLOW_TRACKING_URI=sqlite:///mlflow.db

mlflow models serve -m "models:/churn-model@champion" -p 5001 --env-manager local

# endpoints
GET  /ping         -> 200 when ready        (use as readiness probe)
GET  /version      -> MLflow version
POST /invocations  -> predictions

# ready in ~8 s on a laptop (loads the model, starts uvicorn)
# --env-manager:  local | virtualenv | uv | conda | (docker via build-docker)
```

`--env-manager local` reuses your current Python — fine for class. For real deployments rebuild the pinned environment or use a container.

Calling /invocations — accepted input formats

bash / json

```
# 1. records (one dict per row)  <- we use this one
curl -s localhost:5001/invocations -H 'content-type: application/json' -d \
 '{"dataframe_records":[{"tenure_months":3,"monthly_charge":95.5,"support_calls":5,
   "late_payments":2,"data_usage_gb":20,"contract_months":1,"num_products":1,"age":40}]}'
# -> {"predictions":[{"churn_probability":0.8916,"churn":1,"model_version":"v1"}]}

# 2. split orientation (smaller payload)
{"dataframe_split":{"columns":["tenure_months","..."],"data":[[3,95.5,"..."]]}}

# 3. TF-Serving style            4. tensors / numpy
{"instances":[ {...}, {...} ]}     {"inputs":[[...],[...]]}
```

Schema enforcement vs our hand-written validation (measured)

Case	MLflow (signature)	FastAPI (Deck 1)
missing columns	HTTP 400 — failed to enforce schema	422 — Field required
string in numeric column	HTTP 400 — cannot convert to double	422 — not a valid number
extra, unknown column	HTTP 200 — silently accepted	ignored by the typed model
age = 500 (business rule)	HTTP 200 — no range checks	422 — ≤ 110 rule
error body	{error_code, message}	{detail:[{loc,msg}]} (OpenAPI)

A signature checks types and presence, not business rules. Need ranges or custom routes? Wrap the pyfunc inside your own FastAPI app — a very common pattern.

4

Promote, roll back, batch, containerise

Operating the model, not just running it

Promote and roll back by moving an alias

bash / python

```
# 1. move the alias — metadata only, instantaneous
client.set_registered_model_alias("churn-model", "champion", 2)

# The RUNNING server still answers with v1: the alias is resolved at start-up.

# 2. restart the server so it re-resolves the alias
pkill -f "mlflow models serve"     # wait until the port is really free!
mlflow models serve -m "models:/churn-model@champion" -p 5001 --env-manager local
#    served model_version: v2

# 3. ROLLBACK = same two steps with the old version number
client.set_registered_model_alias("churn-model", "champion", 1)   # + restart

# In Kubernetes the 'restart' is a rolling update: no downtime (Deck 4).
```

Gotcha we hit while building this lecture: a restart can 'succeed' against the OLD process if it hasn't released the port yet — verify model_version in a response, don't trust the ping.

Offline batch scoring — same model, no server

Command: mlflow models predict -m models:/churn-model@champion -i batch_in.json -t json --env-manager local -o batch_out.json

When: nightly scoring, backfills, evaluation on a big file

Why it matters: online vs batch is a deployment decision about latency needs — the model artifact is identical

Scale it: for millions of rows use mlflow.pyfunc.spark_udf (Module 2: Spark)

Containerise: `mlflow models build-docker`

```bash
# build an image that contains the model, its pinned environment and the same server
mlflow models build-docker -m "models:/churn-model@champion" \
      -n churn-mlflow:champion --env-manager virtualenv

docker run -p 5001:8080 churn-mlflow:champion     # container serves on 8080
curl localhost:5001/ping

# into minikube (Deck 4):
minikube -p aiops image load churn-mlflow:champion
```

Needs a Docker daemon (not available in the authoring sandbox — untested here; the notebook prints this command and checks for Docker).

MLflow serve vs hand-written FastAPI — choose deliberately

Dimension	mlflow models serve	FastAPI (Deck 1)
Time to first endpoint	Minutes — one command	Hours — write schema, routes, tests
Validation	Types & presence only	Anything you can code (ranges, rules)
Custom routes / auth	No — wrap it	Full control
Version traceability	Via registry + your pyfunc output	Whatever you return
Per-request overhead	Higher (DataFrame + enforcement)	Lower for small payloads
Measured (1 client)	≈ 73 req/s, p50 12 ms	≈ 147 req/s, p50 6 ms

Same model (v1), same machine. Convenience has a cost — measure for your model rather than assuming.

5

Safe release practices

Promotion gates and shadow testing

A promotion gate (Task A)

```python
import train_models
from sklearn.metrics import accuracy_score
fresh = train_models.make_data(seed=99)        # data neither model has seen

def promotion_gate(min_gain=0.01):
    accs = {}
    for alias in ("champion", "challenger"):
        m = mlflow.pyfunc.load_model(f"models:/churn-model@{alias}")
        pred = m.predict(fresh[train_models.FEATURES].astype("float64"))["churn"]
        accs[alias] = accuracy_score(fresh["churn"], pred)
    return dict(champion_acc=round(accs["champion"], 4),
                challenger_acc=round(accs["challenger"], 4),
                promote=bool(accs["challenger"] - accs["champion"] >= min_gain))

promotion_gate(0.01)   # {'champion_acc': 0.828, 'challenger_acc': 0.8375, 'promote': False}
```

Measured: v2 is better by 0.95 points on fresh data — below our 1-point bar, so it is NOT promoted. (The logged train_accuracy of 0.89 for v2 was in-sample and misleading.)

Shadow test: observe the challenger without trusting it (Task B)

```python
# champion on :5001, challenger on :5002 (two serve processes)
mlflow models serve -m "models:/churn-model@challenger" -p 5002 --env-manager local

lat = {5001: [], 5002: []}; labels = {5001: [], 5002: []}
for rec in rows_all[:200]:
    for port in (5001, 5002):                    # SAME request to both
        t = time.perf_counter()
        r = requests.post(f"http://localhost:{port}/invocations", json={"dataframe_records": [rec]})
        lat[port].append((time.perf_counter() - t) * 1000)
        labels[port].append(r.json()["predictions"][0]["churn"])
dis = np.mean(np.array(labels[5001]) != np.array(labels[5002]))
# measured: disagreement 8.5 %,  p95 latency  champion 16.6 ms  vs  challenger 37.0 ms
```

Clients only ever see the champion's answer. A shadow run reveals disagreement and latency cost before any user is affected (syllabus: deployment patterns — shadow).

Deployment patterns — where each one lives in this module

Pattern	Idea	Where we do it
Shadow	Send copy of traffic to new version; ignore its answers	MLflow notebook, Task B
Canary	Send a small % of real traffic to the new version	Ray Serve router (Deck 3); K8s replica ratio (Deck 4)
Blue-green	Two full stacks; switch the pointer	K8s Service selector patch (Deck 4); MLflow alias flip
Rolling update	Replace instances gradually	Kubernetes Deployment (Deck 4)
Rollback	Return to previous known-good version	Alias move · router weight 0 · `kubectl rollout undo`

HANDS-ON

Notebook 4b — MLflow serving (core ≈ 20 min)

1. §0: write files, train v1/v2, run mlflow_register.py → versions 1 and 2, aliases champion→1, challenger→2
2. §1: inspect the registry and the signature (types of the 8 inputs, 3 outputs)
3. §2: serve the champion on :5001; query /invocations; trigger 3 schema errors — which one is silently accepted?
4. §3: promote → observe the old answer → restart → observe v2 → roll back
5. §6: load-test and compare with your FastAPI numbers
6. Take-home: YOUR TURN A (promotion gate on fresh data) and B (shadow test on :5002)

Notebook: Lecture4b_MLflow_Serving_Exercise.ipynb

Common errors and fixes

Symptom	Likely cause	Fix
`RESOURCE_DOES_NOT_EXIST` for models:/...@champion	Wrong tracking URI (file store vs sqlite)	export MLFLOW_TRACKING_URI=sqlite:///mlflow.db in EVERY shell
Alias moved but old version served	Alias resolved at start-up; or old process still alive	Restart; confirm port is free; read model_version
400 'Failed to enforce schema'	Missing/ill-typed column	Send all 8 features as numbers
Server takes 1+ min to start	env-manager virtualenv/conda rebuilding	Use --env-manager local in class
`Address already in use`	Earlier serve process still running	pkill -f 'mlflow models serve'; pkill -f gunicorn
Telemetry connect errors in logs	Sandbox blocks outbound calls	export MLFLOW_DISABLE_TELEMETRY=true

Recap — MLflow serving

- pyfunc + signature = portable, enforceable serving contract (types, not business rules)
- Registry aliases make promotion and rollback metadata operations; servers resolve them at start-up
- mlflow models serve (dev/test) · build-docker (deploy) · models predict (batch)
- Gate promotions on fresh data; use shadow traffic to compare safely

NEXT → Deck 3 — Ray Serve: replicas, a canary router and autoscaling inside one Python program